LAB SHEET 8

Experiment Name: Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

Steps :

1. Import the required libraries and load a real-world video sequence or capture live video using a webcam.
2. Identify a moving object or select feature points using the Shi-Tomasi Corner Detection algorithm.
3. Apply the Lucas-Kanade Optical Flow algorithm to track the movement of the selected object across consecutive video frames.
4. Draw motion trajectories and displacement vectors to visualize the path followed by the moving object.
5. Calculate the direction and magnitude of object motion and display the tracking results in real time.
6. Repeat the experiment using the Farneback Dense Optical Flow algorithm and compare the tracking performance.
7. Evaluate the impact of object speed, illumination changes, occlusion, and camera movement on tracking accuracy.
8. Analyze the tracked motion to estimate parameters such as object displacement, trajectory, and average speed.
9. Compare the effectiveness of optical flow-based tracking with conventional frame-byframe object detection techniques.
10. Document the observations and discuss practical applications of optical flow in intelligent vision systems.

Solution -> opticalflow.py :

```
import cv2
import numpy as np
import os
import math
VIDEO_PATH = "input.mp4"
os.makedirs("outputs/lucas_kanade", exist_ok=True)
os.makedirs("outputs/farneback", exist_ok=True)
video = cv2.VideoCapture(VIDEO_PATH)
if not video.isOpened():
    print("Error: Could not open video.")
    print("Make sure input_3sec.mp4 exists in the project folder.")
    exit()
fps = video.get(cv2.CAP_PROP_FPS)
total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))
print("=" * 60)
print("OPTICAL FLOW MOTION ESTIMATION")
print("=" * 60)
```

```
print("Video loaded successfully.")
print("Resolution:", width, "x", height)
print("FPS:", fps)
print("Total Frames:", total_frames)
ret, old_frame = video.read()
if not ret:
    print("Error: Could not read first frame.")
    video.release()
    exit()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
old_points = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)
if old_points is None:
    print("No feature points detected.")
    video.release()
    exit()
print("Shi-Tomasi feature points detected:", len(old_points))
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
trajectory_mask = np.zeros_like(old_frame)
frame_count = 0
displacements = []
trajectory_points = []
while True:
    ret, frame = video.read()
    if not ret:
        break
    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )
```

```
new_points, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    old_points,
    None,
    **lk_params
)
if new_points is not None:
    good_new = new_points[status == 1]
    good_old = old_points[status == 1]
    frame_displacements = []
    for new, old in zip(good_new, good_old):
        x_new, y_new = new.ravel()
        x_old, y_old = old.ravel()
        dx = x_new - x_old
        dy = y_new - y_old
        displacement = math.sqrt(
            dx * dx + dy * dy
        )
        frame_displacements.append(displacement)
        displacements.append(displacement)
        trajectory_mask = cv2.line(
            trajectory_mask,
            (int(x_old), int(y_old)),
            (int(x_new), int(y_new)),
            (255, 255, 255),
            2
        )
        frame = cv2.circle(
            frame,
            (int(x_new), int(y_new)),
            4,
            (0, 255, 0),
            -1
        )
        cv2.arrowedLine(
            frame,
            (int(x_old), int(y_old)),
            (int(x_new), int(y_new)),
            (0, 0, 255),
            1,
            tipLength=0.3
        )
    output = cv2.add(
        frame,
        trajectory_mask
    )
    if len(frame_displacements) > 0:
```

```
    average_displacement = np.mean(
        frame_displacements
    )
    if fps > 0:
        average_speed = (
            average_displacement * fps
        )
    else:
        average_speed = 0
else:
    average_displacement = 0
    average_speed = 0
if len(good_new) > 0:
    dx_total = 0
    dy_total = 0
    for new, old in zip(
        good_new,
        good_old
    ):
        x_new, y_new = new.ravel()
        x_old, y_old = old.ravel()
        dx_total += x_new - x_old
        dy_total += y_new - y_old
    dx_average = dx_total / len(good_new)
    dy_average = dy_total / len(good_new)
    angle = math.degrees(
        math.atan2(
            dy_average,
            dx_average
        )
    )
    if angle < 0:
        angle += 360
else:
    angle = 0
if -22.5 <= angle < 22.5:
    direction = "Right"
elif 22.5 <= angle < 67.5:
    direction = "Down-Right"
elif 67.5 <= angle < 112.5:
    direction = "Down"
elif 112.5 <= angle < 157.5:
    direction = "Down-Left"
elif 157.5 <= angle < 202.5:
    direction = "Left"
elif 202.5 <= angle < 247.5:
    direction = "Up-Left"
elif 247.5 <= angle < 292.5:
```

```
        direction = "Up"
    else:
        direction = "Up-Right"
    cv2.putText(
        output,
        "Lucas-Kanade Optical Flow",
        (20, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 255),
        2
    )
    cv2.putText(
        output,
        f"Tracked Points: {len(good_new)}",
        (20, 60),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )
    cv2.putText(
        output,
        f"Displacement: {average_displacement:.2f} pixels",
        (20, 90),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )
    cv2.putText(
        output,
        f"Speed: {average_speed:.2f} pixels/sec",
        (20, 120),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )
    cv2.putText(
        output,
        f"Direction: {direction}",
        (20, 150),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )
```

```
        if frame_count % 15 == 0:
            filename = (
                "outputs/lucas_kanade/"
                f"frame_{frame_count}.jpg"
            )
            cv2.imwrite(
                filename,
                output
            )
        old_gray = frame_gray.copy()
        old_points = good_new.reshape(
            -1,
            1,
            2
        )
    frame_count += 1
video.release()
print()
print("Lucas-Kanade tracking completed.")
print("Lucas-Kanade output saved.")
print()
print("Starting Farneback Dense Optical Flow...")
video = cv2.VideoCapture(VIDEO_PATH)
if not video.isOpened():
    print("Error: Could not reopen video.")
    exit()
ret, old_frame = video.read()
if not ret:
    print("Error: Could not read video.")
    video.release()
    exit()
old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)
frame_count = 0
dense_displacements = []
while True:
    ret, frame = video.read()
    if not ret:
        break
    new_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        new_gray,
```

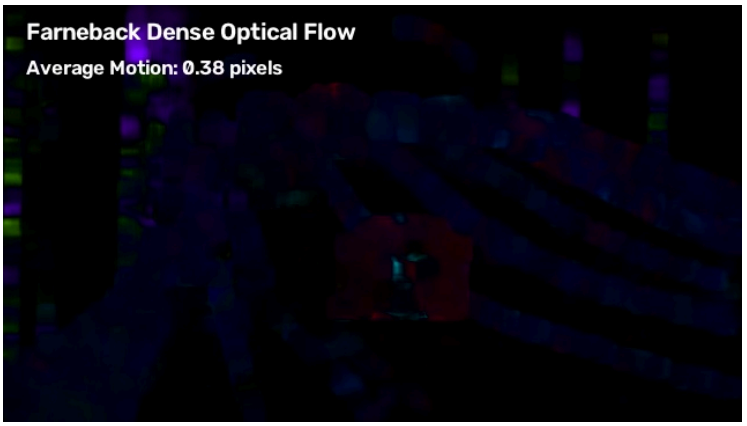
```
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)
average_magnitude = np.mean(
    magnitude
)
dense_displacements.append(
    average_magnitude
)
hsv = np.zeros_like(frame)
hsv[..., 0] = (
    angle * 180 / np.pi / 2
)
hsv[..., 1] = 255
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)
dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)
cv2.putText(
    dense_flow,
    "Farneback Dense Optical Flow",
    (20, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2
)
cv2.putText(
    dense_flow,
    f"Average Motion: {average_magnitude:.2f} pixels",
    (20, 60),
```

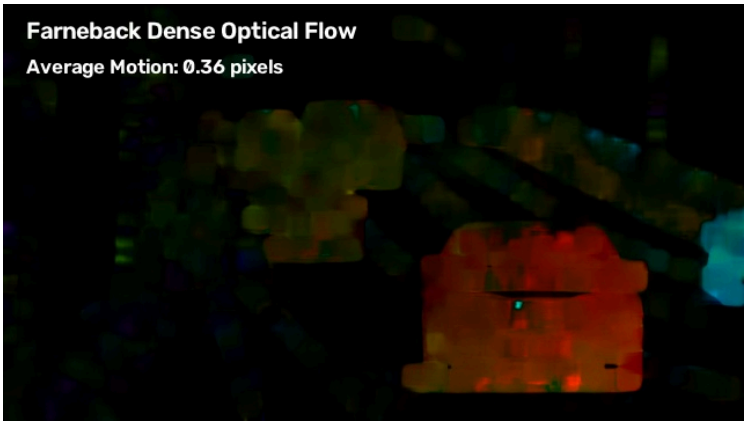
```
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
    if frame_count % 15 == 0:
        filename = (
            "outputs/farneback/"
            f"frame_{frame_count}.jpg"
        )
        cv2.imwrite(
            filename,
            dense_flow
        )
    old_gray = new_gray.copy()
    frame_count += 1
video.release()
print()
print("=" * 60)
print("MOTION ANALYSIS")
print("=" * 60)
if len(displacements) > 0:
    total_displacement = sum(
        displacements
    )
    average_displacement = np.mean(
        displacements
    )
    maximum_displacement = np.max(
        displacements
    )
    if fps > 0:
        average_speed = (
            average_displacement * fps
        )
    else:
        average_speed = 0
    print(
        f"Total tracked displacement: "
        f"{total_displacement:.2f} pixels"
    )
    print(
        f"Average displacement: "
        f"{average_displacement:.2f} pixels/frame"
    )
    print(
        f"Maximum displacement: "
        f"{maximum_displacement:.2f} pixels/frame"
```

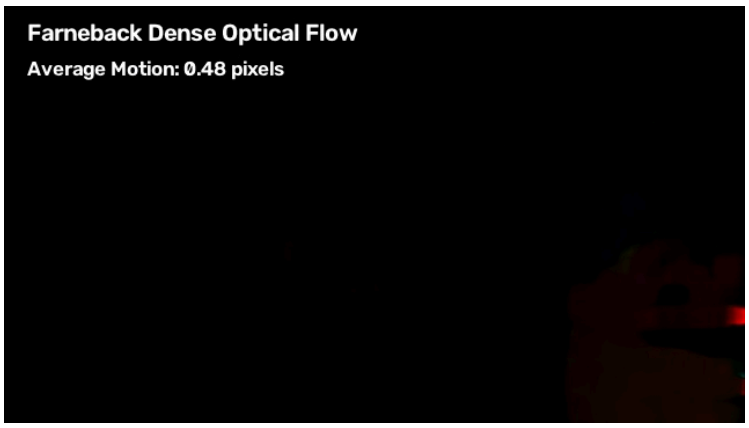
```
    )
    print(
        f"Average speed: "
        f"{average_speed:.2f} pixels/second"
    )
if len(dense_displacements) > 0:
    dense_average = np.mean(
        dense_displacements
    )
    print(
        f"Farneback average motion: "
        f"{dense_average:.2f} pixels"
    )
print()
print("=" * 60)
print("METHOD COMPARISON")
print("=" * 60)
print(
    "Optical Flow:"
)
print(
    "- Tracks motion between consecutive frames."
)
print(
    "- Preserves trajectory information."
)
print(
    "- Lucas-Kanade tracks selected feature points."
)
print(
    "- Farneback estimates dense pixel-level motion."
)
print()
print(
    "Conventional Frame-by-Frame Detection:"
)
print(
    "- Detects objects independently in each frame."
)
print(
    "- Does not inherently provide motion vectors."
)
print(
    "- Additional tracking is usually required."
)
print()
print("=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
```

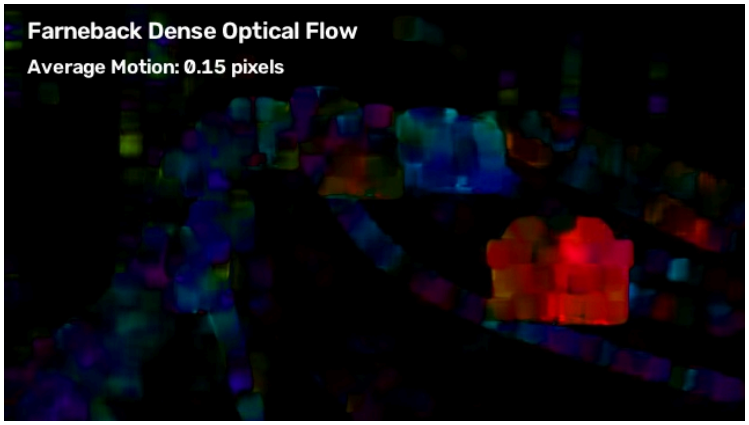
```
print("=" * 60)
print("Lucas-Kanade results:")
print("outputs/lucas_kanade/")
print()
print("Farneback results:")
print("outputs/farneback/")
print()
print("Observations:")
print("1. Shi-Tomasi detects suitable feature points.")
print("2. Lucas-Kanade tracks selected feature points.")
print("3. Motion trajectories show object movement.")
print("4. Displacement represents movement in pixels.")
print("5. Direction is calculated from displacement vectors.")
print("6. Farneback provides dense motion estimation.")
print("7. Illumination and occlusion can affect tracking.")
print("8. Camera movement can produce large optical-flow vectors.")
print("9. Optical flow provides useful motion information.")
print("10. Optical flow is useful in intelligent vision systems.")
```

Output -> farneback :

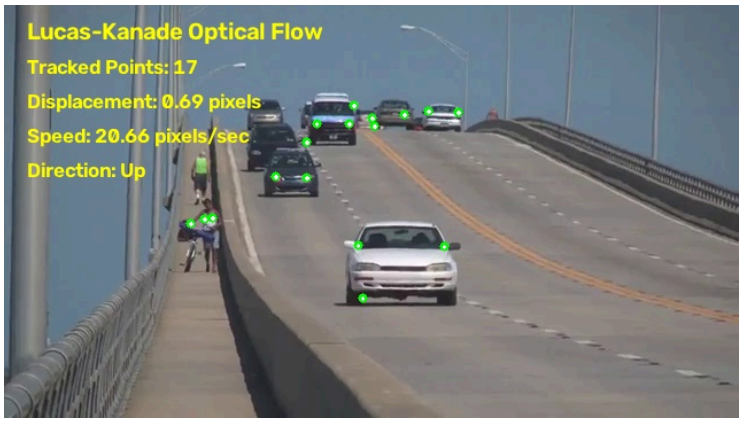


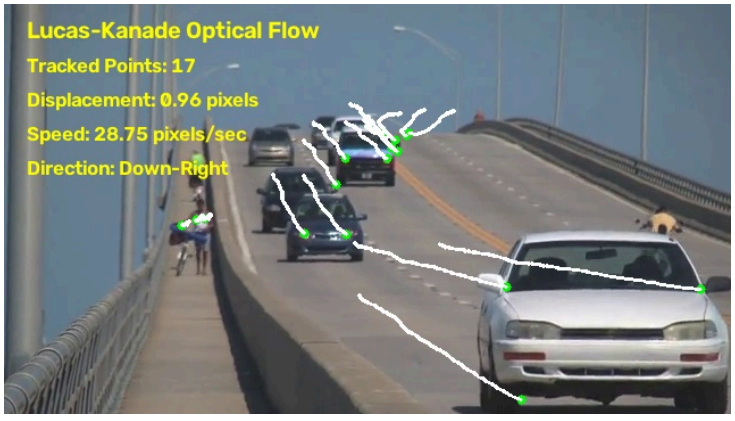


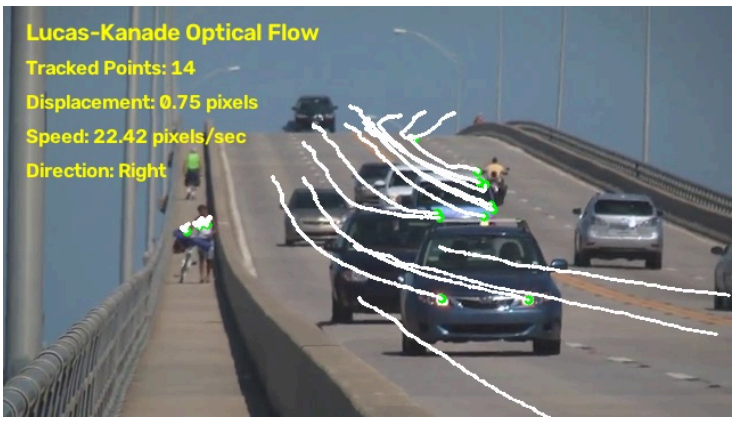


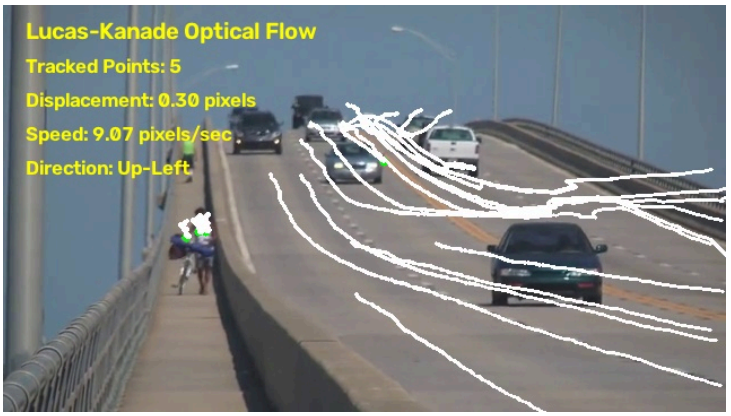


lucas_kanade :






Lucas-Kanade Optical Flow
Tracked Points: 14
Displacement: 0.75 pixels
Speed: 22.42 pixels/sec
Direction: Right


Lucas-Kanade Optical Flow
Tracked Points: 5
Displacement: 0.30 pixels
Speed: 9.07 pixels/sec
Direction: Up-Left